

04 Console and console methods

Console and Console Methods in JavaScript - Complete Lab Guide



Introduction

The **console** is a global object in JavaScript that allows you to print logs, warnings, errors, and various types of messages. This is an essential tool for debugging and understanding how your code works.



Setup Instructions

1. **Open VS Code**
2. **Create a new folder** called `js-console-practice`
3. **Create a new file** called `console.js`
4. **Open the Terminal** in VS Code (View → Terminal or `Ctrl+``)



Chapter 1: Basic Console Methods

1.1 `console.log()` - Standard Output

Purpose: Prints general information to the console.

Step 1: Type the following code in `console.js`:

```
console.log("Hello World");
```

JS

Step 2: Run your code in two ways:

Method A - Using VS Code:

- Press `Ctrl+F5` (or `Cmd+F5` on Mac) or go to **Run → Run Without Debugging**
- Check the **Debug Console** (View → Debug Console)

Method B - Using Terminal:

```
node console.js
```

SHELL

Expected Output:

```
Hello World
```

1.2 Console.error() - Error Messages

Purpose: Displays error messages in **red color** for easy identification.

Step 3: Add this code below your previous line:

```
console.log("Hello World");  
console.error("Array error occurred");
```

JS

Step 4: Run the code again.

Expected Output:

```
Hello World  
Array error occurred    (in red color)
```

1.3 Console.warn() - Warning Messages

Purpose: Displays warning messages (usually in yellow/orange).

Step 5: Add this line:

```
console.log("Hello World");  
console.error("Array error occurred");  
console.warn("This is a warning message");
```

JS

Step 6: Run the code.

Expected Output:

```
Hello World  
Array error occurred    (in red)  
This is a warning message (in yellow/orange)
```

1.4 Console.info() - Information Messages

Purpose: Displays informational messages.

Step 7: Add this line:

```
console.log("Hello World");  
console.error("Array error occurred");  
console.warn("This is a warning message");  
console.info("This is my JS code");
```



Chapter 2: Advanced Console Methods

2.1 Console.table() - Display Data in Tables

Purpose: Displays objects and arrays in a beautiful table format.

Step 8: Clear your file and start fresh with this code:

```
// Creating a user object  
const user = {  
  name: "John",  
  age: 25,  
  city: "New York"  
};  
  
// Normal log  
console.log(user);  
  
// Table format  
console.table(user);
```

Step 9: Run the code.

Expected Output:

```
{ name: 'John', age: 25, city: 'New York' }
```

(index)	Values
name	'John'
age	25
city	'New York'

2.2 Console.table() - Multiple Objects

Step 10: Modify your code to handle two objects:

```
JS
// Creating two user objects
const user = {
  name: "John",
  age: 25,
  city: "New York",
  country: "USA"
};

const user1 = {
  name: "Lisa",
  age: 30,
  city: "LA",
  country: "USA"
};

// Display both users in table format
console.table({user, user1});
```

Step 11: Run the code.

Expected Output: A nicely formatted table with both users' data.



Chapter 3: Organizing Console Output

3.1 Console.group() - Grouping Logs

Purpose: Groups related console messages together with collapsible sections.

Step 12: Clear your file and add:

```
JS
// Start a group
console.group("Login Feature");

console.log("Enter username");
console.log("Enter password");
console.log("Click submit button");
console.log("Redirect to dashboard");

// End the group
console.groupEnd();
```

Step 13: Run the code.

Expected Output:

```
▼ Login Feature
  Enter username
  Enter password
  Click submit button
  Redirect to dashboard
```

💡 **Tip:** You can click the arrow to collapse/expand the group!

Chapter 4: Performance Measurement

4.1 `Console.time()` - Measuring Execution Time

Purpose: Measures how long a code block takes to execute.

Step 14: Clear your file and add:

```
// Start the timer
console.time("Print 1 to N values");

// Task: Print numbers from 1 to 10000
for(let i = 1; i ≤ 10000; i++) {
  console.log(i);
}

// End the timer
console.timeEnd("Print 1 to N values");
```

JS

Step 15: Run the code.

Expected Output:

```
1
2
3
...
10000
Print 1 to N values: 1058.282ms
```

Step 16: Try with 100,000 iterations:

JS

```
console.time("Print 1 to N values");

for(let i = 1; i ≤ 100000; i++) {
    console.log(i);
}

console.timeEnd("Print 1 to N values");
```

Expected Output: You'll see it takes longer (around 7-8 seconds).



Chapter 5: Utility Console Methods

5.1 Console.clear() - Clearing the Console

Purpose: Clears all previous console output.

Step 17: Try this code:

JS

```
console.log("This will be cleared");
console.log("This will also be cleared");

// Clear the console
console.clear();

console.log("Console was cleared - only this appears");
```

5.2 Console.count() - Counting Method Calls

Purpose: Counts how many times a specific label has been logged.

Step 18: Add this code:

JS

```
console.count("Reset password");
console.count("Reset password");
console.count("Reset password");
```

Step 19: Run the code.

Expected Output:

```
Reset password: 1  
Reset password: 2  
Reset password: 3
```

5.3 Console.assert() - Conditional Error Messages

Purpose: Prints an error message only if the condition is **false**.

Step 20: Try this code:

```
// This will print error (10 is not equal to 20)  
console.assert(10 === 20, "This is failed - assertion error");  
  
// This will NOT print anything (10 equals 10)  
console.assert(10 === 10, "This is failed - assertion error");  
  
// Using with variables  
let flag = false;  
console.assert(flag, "Flag is false - assertion failed");  
  
flag = true;  
console.assert(flag, "This won't print because flag is true");
```

JS

Expected Output:

```
Assertion failed: This is failed - assertion error  
Assertion failed: Flag is false - assertion failed
```

5.4 Console.trace() - Stack Trace

Purpose: Shows the execution path of a function.

Step 21: Add this code:

```
function testMyJobFunction() {  
    console.trace("Tracing this function");  
}  
  
// Call the function  
testMyJobFunction();
```

JS

Step 22: Run the code.

Expected Output:

```
Trace: Tracing this function
  at testMyJobFunction (console.js:2:13)
  at Object.<anonymous> (console.js:6:1)
  ...
```

Chapter 6: Styling Console Output

6.1 Adding CSS Styles to Console Messages

Purpose: Makes your console output colorful and eye-catching.

Step 23: Use this code:

```
console.log(  
  "%cI love JavaScript coding",  
  "color: red; background: green; border: solid red; font-size: 20px;"  
);
```

JS

Step 24: Run the code.

Expected Output: Your message appears with red text, green background, and red border!

6.2 Experimenting with Different Styles

Step 25: Try different color combinations:

```
console.log(
  "%cI love JavaScript coding",
  "color: lightblue; background: red; border: solid green; font-size: 20px;
padding: 10px;"
);

console.log(
  "%cWelcome to My App",
  "color: white; background: purple; font-weight: bold; font-size: 24px; padding:
15px;"
);

console.log(
  "%cError Message",
  "color: red; background: yellow; font-weight: bold; border: 3px dashed red;
padding: 10px;"
);
```

Chapter 7: Testing in Browser Console

7.1 Creating an HTML File

Step 26: Create a new file called `index.html`:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Console Methods Demo</title>
</head>
<body>
  <h1>Open the Console (F12) to see outputs</h1>
  <script src="console.js"></script>
</body>
</html>
```

Step 27: Right-click on `index.html` and select "Open with Live Server"

Step 28: Press **F12** to open Developer Tools and go to the **Console** tab

Step 29: You'll see all your console outputs in the browser!

Complete Practice Exercise

Step 30: Create a comprehensive example combining everything:

```
// === COMPLETE CONSOLE METHODS DEMO ===

console.log("%c=== Console Methods Demo ===",
  "color: white; background: blue; font-size: 20px; padding: 10px; font-weight: bold;"
);

// 1. Basic Logging
console.group("1. Basic Logging Methods");
console.log("Standard log message");
console.info("Information message");
console.warn("Warning message");
console.error("Error message");
console.groupEnd();

// 2. Objects in Table
console.group("2. Displaying Objects");
const employee1 = { name: "Alice", role: "Developer", salary: 80000 };
const employee2 = { name: "Bob", role: "Designer", salary: 70000 };
console.table({ employee1, employee2 });
console.groupEnd();

// 3. Counting
console.group("3. Counting Operations");
for(let i = 0; i < 5; i++) {
  console.count("Loop iteration");
}
console.groupEnd();

// 4. Timing
console.group("4. Performance Timing");
console.time("Array operation");
let numbers = [];
for(let i = 0; i < 1000; i++) {
  numbers.push(i);
}
console.timeEnd("Array operation");
console.groupEnd();

// 5. Assertions
console.group("5. Assertions");
console.assert(5 > 3, "This won't show - assertion passed");
console.assert(5 < 3, "Assertion failed: 5 is not less than 3");
console.groupEnd();
```

```
// 6. Function Tracing
console.group("6. Function Tracing");
function level1() {
    level2();
}
function level2() {
    level3();
}
function level3() {
    console.trace("Call stack trace");
}
level1();
console.groupEnd();

console.log("%c=== Demo Complete! ===",
    "color: green; background: lightgreen; font-size: 18px; padding: 10px; font-weight: bold;");
```



Quick Reference Cheat Sheet

Method	Purpose	Example
<code>console.log()</code>	Standard output	<code>console.log("Hello")</code>
<code>console.error()</code>	Error messages (red)	<code>console.error("Error!")</code>
<code>console.warn()</code>	Warnings (yellow)	<code>console.warn("Warning!")</code>
<code>console.info()</code>	Information	<code>console.info("Info")</code>
<code>console.table()</code>	Display as table	<code>console.table(object)</code>
<code>console.group()</code>	Start group	<code>console.group("Name")</code>
<code>console.groupEnd()</code>	End group	<code>console.groupEnd()</code>
<code>console.time()</code>	Start timer	<code>console.time("Task")</code>
<code>console.timeEnd()</code>	End timer	<code>console.timeEnd("Task")</code>
<code>console.clear()</code>	Clear console	<code>console.clear()</code>
<code>console.count()</code>	Count calls	<code>console.count("Label")</code>
<code>console.assert()</code>	Conditional error	<code>console.assert(condition, msg)</code>
<code>console.trace()</code>	Stack trace	<code>console.trace()</code>
<code>%c</code>	CSS styling	<code>console.log("%cText", "color: red")</code>



Key Takeaways

1. **`console.log()`** is great for general debugging

2. **console.table()** makes objects readable
 3. **console.group()** organizes related logs
 4. **console.time()** measures performance
 5. **console.assert()** validates conditions
 6. **CSS styling** makes important messages stand out
-

✓ Practice Challenges

Challenge 1: Create a function that logs user activity with different console methods based on the action type (login = info, logout = warn, error = error).

Challenge 2: Build a simple calculator and use `console.time()` to measure which operation (add/subtract/multiply/divide) is fastest.

Challenge 3: Create a styled console welcome message for a fictional app using CSS styling.

Happy Coding! 🚀